UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Systems Analysis & Design — Workshop No. 2

Computer Engineering Program | Semester 2026-I

Smart Campus Parking Management System

System Design Document

Kevin Zambrano Matiz	20251020118
Dilan Santiago Herreño Cifuentes	20251020117
Ronaldo Andrés Alvarado Doria	20251020122
Esteban Josué Varela Sarmiento	20251020114

Bogotá, D.C. — 2026

Executive Summary

This document presents the complete System Design for the Smart Campus Parking Management System (SCPMS), a proposed digital solution for the motorcycle parking facility at Basement 3 of Universidad Distrital Francisco José de Caldas. The design builds directly on the findings of Workshop #1, which confirmed through direct observation and surveys that the facility's inefficiency stems from three root causes: *information asymmetry* (users do not know slot availability before arriving), *spatial disorder* (car markings cause a single motorcycle to occupy three spaces), and *absence of prioritization* (no mechanism to align occupancy with academic schedules).

The proposed solution is a software-first, phased platform integrated into the university's existing mobile application. Its five core modules — Reservation & Availability, Time-Slot Scheduling Engine, User Account Integration, Security Alert Subsystem, and External Overflow Advisory Module — collectively eliminate all three root causes without requiring physical infrastructure investment in the first three phases. Phase 4 optionally adds IoT sensors for full automation.

The design is governed by four engineering principles: modularity, scalability, compliance-by-design (Colombian Law 1581 of 2012), and phased delivery. All four specific system objectives — reducing check-in time, increasing space utilization by 20%, achieving 100% Habeas Data compliance, and automating behavioural enforcement — are addressed by concrete design decisions with measurable acceptance criteria.

1. Analysis Review and Requirements Definition

This section synthesises the critical findings from Workshop #1 and translates them into measurable design requirements for the SCPMS. The analysis confirmed that the core problem in Basement 3 is informational and organisational — not a lack of physical space — making a software-first intervention both sufficient and appropriate.

1.1 Critical Constraints

- **Fixed Capacity:** The system operates on a fixed physical resource. Efficiency gains come from algorithmic optimisation, not infrastructure expansion.
- **Mandatory Interoperability:** SCPMS must operate within the university's existing ecosystem — authentication, schedule database, and mobile app.
- **Network Dependency:** Basement 3's underground location may cause connectivity degradation, requiring offline contingency design.
- **Power Supply:** A power failure renders the system unusable. Backup procedures must be defined.
- **Usage Dependency:** The system requires broad adoption by motorcyclists. If a significant portion parks without using the app, the reservation model breaks down.
- **Data Protection (Habeas Data):** The system collects sensitive behavioural data

subject to Colombian Law 1581 of 2012.

1.2 Performance Bottlenecks

- **Check-in/Check-out Queues:** Validation is not instantaneous; queues can form at the access ramp during peak hours.
- **Data Latency:** Unstable connections in the basement may re-introduce the information asymmetry the system is designed to eliminate.
- **Human Intervention Gaps:** No-shows block reserved spaces; overstays delay access for subsequent users.
- **Identity Synchronisation:** The system inherits weaknesses from the university's central authentication service.

1.3 Optimisation Opportunities

- **Automatic Sanctions:** Prevent repeat violators from using the app for a defined period.
- **Dynamic TTL Release:** Automatically free a reserved slot if no check-in occurs within 20 minutes of the reservation start time.

1.4 Measurable Design Requirements

Performance

- Check-in transaction completed in under 3 minutes end-to-end.
- Slot map status updates reflected in the app in under 3 seconds.
- Backend processes concurrent reservation requests without response time degradation.

Reliability

- System availability $\geq 99\%$ during university operating hours (06:00–22:00 weekdays).
- Offline contingency mode: security staff can access a cached reservation list when connectivity is lost.

Scalability

- Database supports up to 10,000 active motorcycle users without query performance loss.
- Integration with the university's SSO (Single Sign-On) for identity validation.
- Modular architecture allows adding new parking areas without rewriting core logic.

Usability & Compliance

- Reservation flow completable in a minimal number of steps; accidental reservations are correctable.

- 20-minute TTL: reservations auto-expire if no check-in occurs, freeing the slot automatically.
- All personally identifiable data encrypted and subject to automatic purge protocols per Law 1581 of 2012.

1.5 Stakeholder Needs and System Objectives

Table 1: Stakeholder Needs and System Requirements

tableheader		
Students	Certainty of availability and fast access	Low-click reservation interface; real-time status updates
Security Staff	Conflict reduction; minimised manual validation	Live map panel; automated violation alerts; incident logging
Parking Administration	Maximise space turnover in Basement 3	Auto-release algorithm (20-min TTL); occupancy analytics
University Management	Data-driven decisions; institutional order	Analytics dashboard: peak hours, reservation volume, utilisation rate

General Objective

Implement an algorithmic management system that eliminates disorder in Basement 3 by replacing unstructured, first-come-first-served occupation with a structured, automated reservation model that maximises the use of fixed physical capacity.

Specific Objectives

1. **Operational Optimisation:** Reduce average check-in time to under 3 minutes.
2. **Inventory Efficiency:** Increase effective space utilisation by at least 20% through dynamic slot status management (Free / Reserved / Occupied).
3. **Regulatory Compliance:** Ensure 100% adherence to Habeas Data law by anonymising and auto-purging student mobility records.
4. **Behavioural Control:** Implement an automatic sanctions mechanism triggered by reservation violations.

2. System Architecture

The SCPMS is designed as a modular extension of the university's existing mobile application, not a standalone product. The architecture separates concerns across five core modules, each with well-defined responsibilities and interfaces. Three external systems are federated: the

Identity Management System, the Academic Schedule Database, and the Campus Security Communication Infrastructure.

2.1 High-Level Architecture

The diagram below illustrates the system boundary, component relationships, data flows, and integration points. External actors interact exclusively through the University Mobile Application gateway. All core SCPMS logic runs within the system boundary, with data persisted in federated university systems rather than external cloud services — satisfying the Habeas Data compliance constraint.

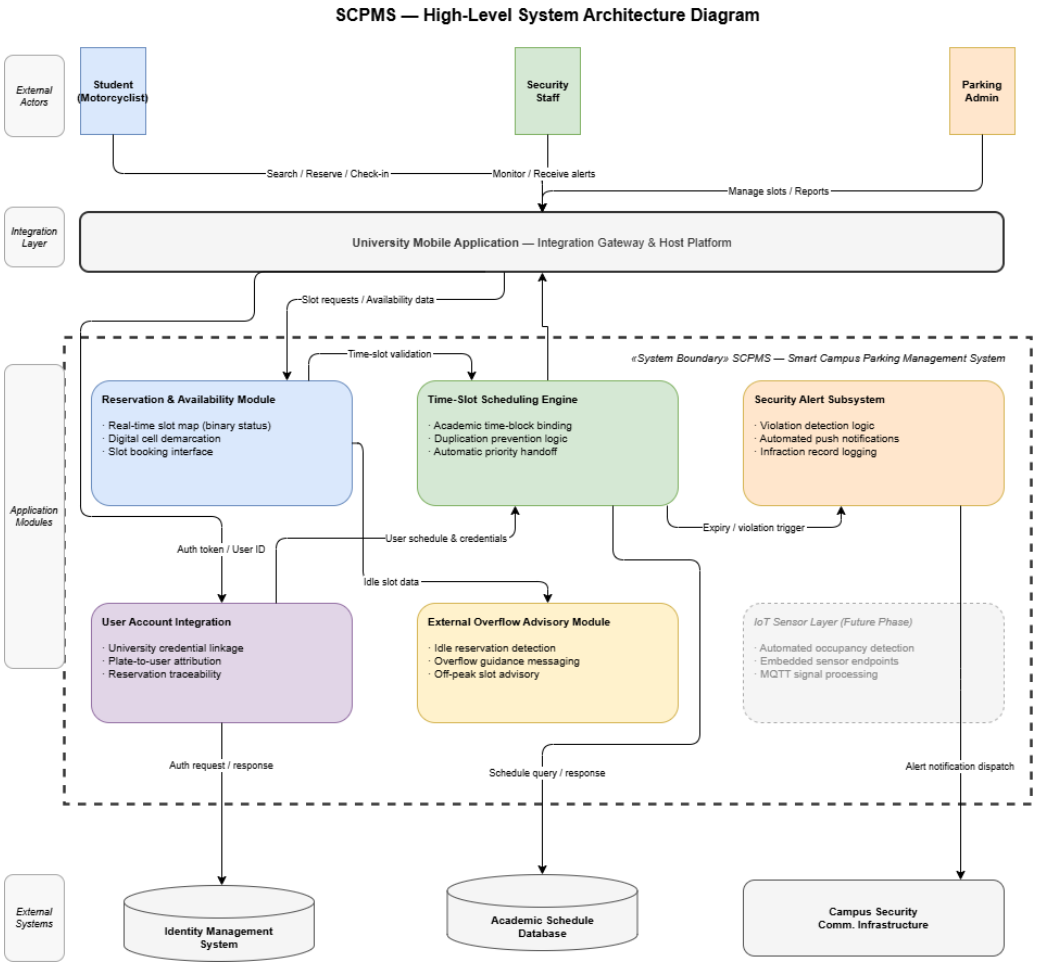

Figure 1: SCPMS — High-Level System Architecture Diagram

2.2 Core Module Responsibilities

Table 2: Core Module Responsibilities and Key Interfaces

tableheader		
Reservation & Availability	Real-time numbered slot map (binary status); slot booking interface; digital cell demarcation	REST API → Mobile App; WebSocket → live map updates
Time-Slot Scheduling Engine	Academic time-block binding; duplication prevention logic; automatic priority handoff at slot expiry	SQL query → Academic Schedule DB; internal event → Security Alert Subsystem
User Account Integration	University credential linkage; plate-to-user attribution; reservation traceability	OAuth 2.0 → Identity Management System
Security Alert Subsystem	Violation detection logic; automated FCM push notifications to security staff; infraction record logging	FCM Push API → Campus Security Comm. Infrastructure
External Overflow Advisory Module	Idle reservation detection; overflow guidance messaging; off-peak slot advisory for external users	In-App Push / SMS → External Parking Users

2.3 Interface & Integration Diagram

The interface diagram details the protocols and data formats used at each integration point. REST API handles slot availability, reservation requests, and status updates between the mobile app and the SCPMS backend. OAuth 2.0 manages authentication against the university's Identity Management System. SQL queries retrieve user schedule data from the Academic Schedule Database. Firebase Cloud Messaging dispatches real-time push alerts to security staff.

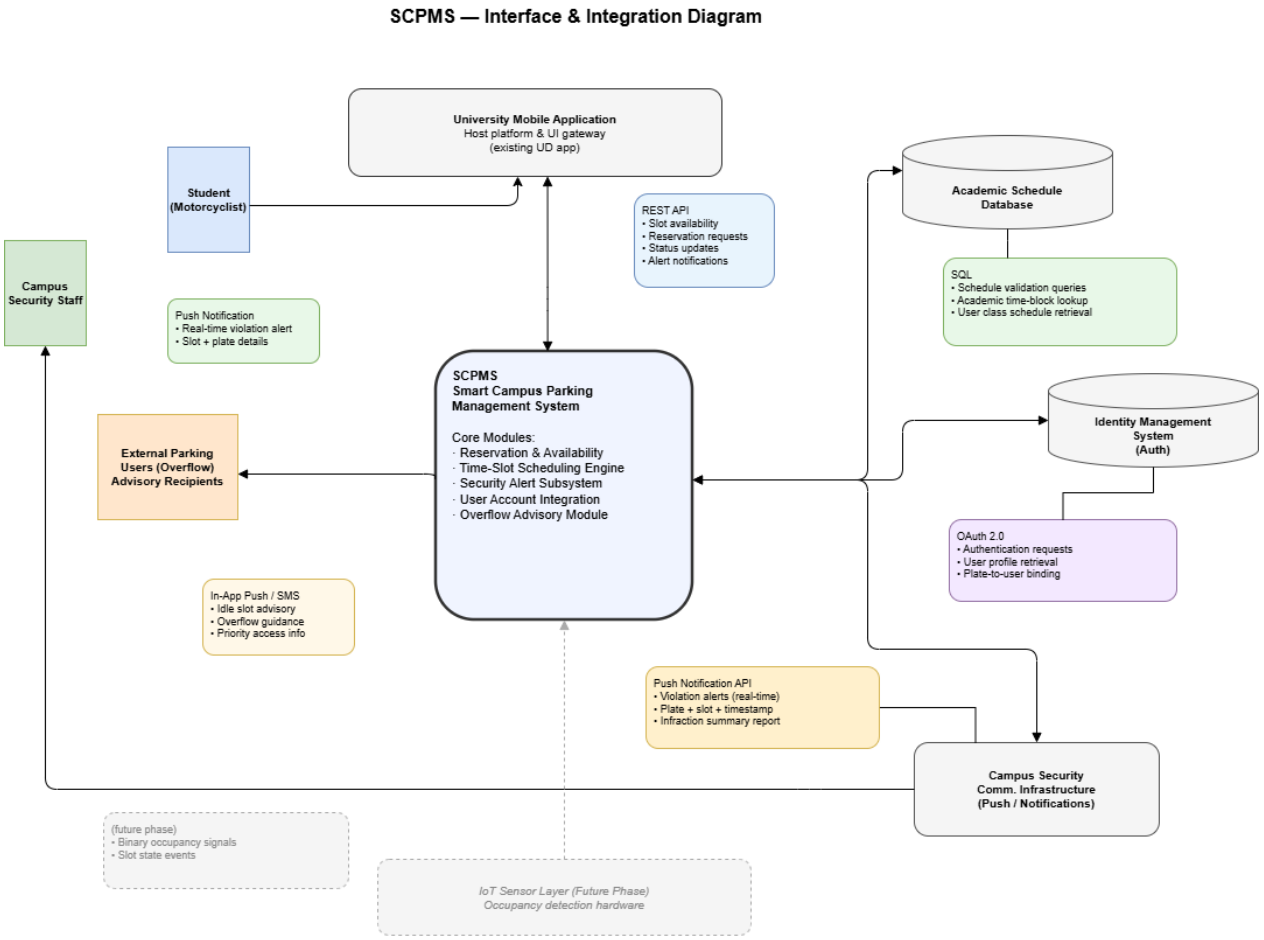

Figure 2: SCPMS — Interface & Integration Diagram

2.4 Process Flow

The operational flow is organised into three phases: Reservation (steps 1–11), Arrival & Check-in (steps 12–13), and Checkout (steps 14–16). A parallel Security Violation Subprocess is triggered automatically when a reserved slot is occupied by an unregistered vehicle or when a user exceeds their time window, escalating to the Security Alert Subsystem and updating slot status to `VIOLATION_PENDING`.

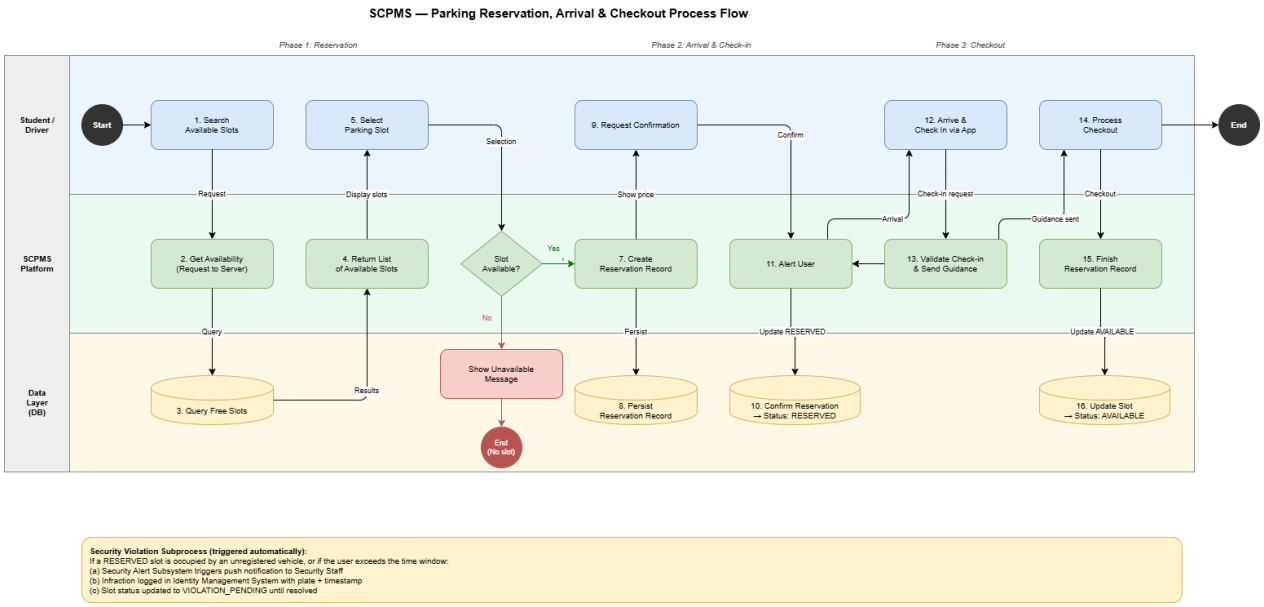

Figure 3: SCPMS — Parking Reservation, Arrival & Checkout Process Flow

2.5 Systems Engineering Principles Applied

- **Modularity:** Each module has a single, well-defined responsibility. Changes to the Scheduling Engine do not require modifications to the Reservation or Security modules.
- **Scalability:** The PostgreSQL schema and NestJS microservice structure support adding new parking areas by extending configuration, not rewriting core logic.
- **Reliability:** Offline contingency mode, transactional locks for concurrent reservations, and Redis-based state caching reduce single points of failure.
- **Maintainability:** All external dependencies (university systems) have documented fallback procedures if unavailable during development or operation.

3. Technical Implementation Strategy

The implementation follows a phased delivery model in which each phase produces a fully functional, independently valuable increment. This approach reduces project risk, allows early user feedback, and ensures that if the project is paused at any phase, the previous phase continues to operate and provide value.

3.1 Technology Stack

Table 3: Selected Technology Stack

tableheader		
Mobile App	Flutter (Android/iOS)	Single codebase for both platforms; Android dominates student device landscape
Interactive Slot Map	Dynamic SVG (in-app)	Real-time slot colouring (green = free, red = occupied) without screen reload
Server / Business Logic	Node.js + NestJS	Modular architecture (reservations, alerts, users); open-source; well-documented
Database	PostgreSQL	Reliable, open-source; supported by university infrastructure
Live Updates	WebSockets + Redis	Instant map updates without polling; Redis caches slot state in memory for <3 s response
Push Notifications	Firebase Cloud Messaging (FCM)	Free up to 10M messages/month; <5 s delivery to security staff devices
Physical Sensors (Phase 4)	ESP32 + Ultrasonic Sensor	Automated occupancy detection; low cost (≈$15,000 COP/slot)

3.2 Implementation Phases

Table 4: Phased Implementation Plan

tableheader			
Phase 1 — Foundations	4 weeks	UD SSO integration; PostgreSQL schema with Basement 3 slots; first app screens	Student logs in with UD credentials and sees the slot list
Phase 2 — Working App	6 weeks	Live slot map (SVG); full reservation flow (search → select → confirm → check-in → checkout); TTL auto-release; FCM reminders	Reservation completed in <3 min; map updates in <3 s
Phase 3 — Control & Alerts	4 weeks	Security staff panel with live map; automated violation alerts; duplicate reservation prevention; infraction logging	Violation triggers guard alert in <5 s; concurrent reservations architecturally impossible
Phase 4 — Physical Sensors	6 weeks	ESP32 sensor per slot; automated occupancy detection; analytics dashboard for Parking Administration	Sensor accuracy ≥95%; parking search time reduced ≥40% vs. baseline

3.3 University System Integration

Table 5: University System Integration and Fallback Procedures

tableheader		
User System / University ID	Verify that the reservation holder is an active student or faculty member	Separate registration using institutional `@udistrital.edu.co` email
Academic Schedule Database	Link reservations to the user's actual class schedule to prevent unnecessary occupancy	Semester schedule loaded manually as a configuration file at term start
Existing UD Mobile App	Embed the SCPMS parking module inside the app students already use	Deploy SCPMS as a standalone app while integration is arranged with IT

All user data — vehicle licence plates, reservation history, location within campus — is stored exclusively on the university's own servers and never transmitted to external services, in compliance with Colombian Law 1581 of 2012.

3.4 Required Team

Table 6: Project Team Roles and Responsibilities

tableheader		
Technical Coordinator (×1)	20 h/week	Architecture decisions; communication with UD IT department
Backend Developers (×2)	15 h/week each	Reservation logic; university system integrations
Mobile App Developer (×1)	15 h/week	Flutter screens; interactive SVG map; reservation flow
Server & Testing Lead (×1)	10 h/week	PostgreSQL management; deployment; performance testing

4. Risk and Complexity Management

Risk identification is organised into technical and organisational categories. Each risk is assessed by probability and impact, with a prevention strategy and a response plan if the risk materialises.

4.1 Technical Risks

Table 7: Technical Risk Assessment

tableheader				
No straightforward API to query academic schedules	High	High	Contact IT from Sprint 1 to explore available options	Load schedule manually as a file at semester start
Concurrent reservation conflict (two users, same slot)	Medium	Critical	Transactional lock in DB: one reservation per slot at a time	Load testing pre-launch; clear error message to second requester
Slot map loads slowly during peak hours (06:30–08:30)	Medium	Medium	Redis in-memory cache for slot state; <3 s response target	App shows last saved state with query timestamp if live update fails
Insufficient Wi-Fi coverage in Basement 3	Low	Medium	Signal strength test before Phase 2 deployment	Offline check-in registered locally and synced when signal is restored
Existing UD app does not allow new module integration	Medium	High	Design SCPMS as standalone app from the start; integration is an enhancement, not a requirement	Distribute SCPMS as a direct APK install among students

4.2 Organisational Risks

Table 8: Organisational Risk Assessment

tableheader				
Students do not adopt the system out of habit	Medium	High	Classroom demos; awareness campaign; direct benefit is clear (guaranteed spot)	4-week voluntary pilot before enforcing mandatory use for basement access
IT department does not authorise server deployment	Medium	Critical	Involve IT from Phase 1; all data stays within UD; present security review	Use a research group server or escalate to the Academic Vice-Rectory
Security staff does not adopt the alert panel	Medium	Medium	2-hour training session; simple interface; alerts sent to phone	Also dispatch alerts to the security department's institutional email as backup

4.3 Complexity Management Strategies

- **Transactional integrity:** PostgreSQL transactional locks prevent duplicate reservations under any concurrency scenario, directly addressing the identified race-condition bottleneck.
- **State resilience:** Redis caches the last known slot state. If the WebSocket connection drops, the app displays the cached state with a timestamp rather than showing stale or blank data.
- **Offline fallback:** Security staff retain access to a downloaded reservation list for manual validation during connectivity loss, preserving minimum operational capacity.
- **Phased adoption:** A 4-week voluntary pilot period before mandatory enforcement allows the system to build user trust and identify UX friction points before full rollout.
- **Compliance by design:** Data minimisation, server-side encryption, and automatic purge protocols are architectural decisions, not post-deployment additions — eliminating the risk of regulatory non-compliance.

5. Performance and Quality Assurance

This section defines the measurable performance targets, monitoring approach, and validation strategy for the SCPMS. All metrics are traceable to the design requirements in Section 1 and to the operational problems documented in Workshop #1.

5.1 Key Performance Indicators

Table 9: Key Performance Indicators

tableheader			
Slot availability query latency	<2 s (95th pct.)	API response logs + Redis hit rate	Phase 2
Check-in transaction time	<3 min end-to-end	Timestamp delta: arrival → access granted	Phase 2
Slot map update delay	<3 s after status change	WebSocket event latency tracking	Phase 2
Concurrent reservation processing	No degradation under peak load	Load test results (06:30–08:30 scenario)	Phase 2
Space utilisation rate	≥20% improvement vs. baseline	Daily occupancy analytics (occupied / total slots)	Phase 4
Reservation no-show rate	<15% of total reservations	TTL auto-release log analysis	Phase 3
Security alert dispatch time	<5 s after violation trigger	FCM delivery receipts	Phase 3
Sensor detection accuracy	≥95% correct detection	Weekly hardware validation audit	Phase 4
System availability (uptime)	≥99% during operating hours	Health-check endpoint + uptime monitor	Phase 1
User error rate (reservation flow)	<5% incomplete or accidental reservations	Failed/cancelled reservation logs	Phase 2

5.2 Monitoring Strategy

The monitoring stack reuses the tools already selected in the implementation strategy (Node.js, PostgreSQL, Redis, FCM, WebSockets), avoiding additional infrastructure overhead.

Table 10: Monitoring Strategy by Dimension

tableheader			
API response time & latency	NestJS interceptor middleware	Admin dashboard	Continuous
Slot map update delay	WebSocket event timestamps vs. DB write	Latency log	Continuous
Redis cache performance	Redis INFO stats (hit/miss rate)	Redis panel	Continuous
System availability	Health-check endpoint + external uptime monitor	Alert to IT (email/SMS)	Real-time
Security violation events	Security Alert Subsystem infraction log	Security staff panel + daily report	Per-event + daily
Space utilisation rate	Reservation records + TTL release events in PostgreSQL	Admin dashboard chart	Daily / weekly
User behaviour analytics	In-app event tracking (reservation funnel, error events)	Admin analytics module	Weekly
Phase 4 sensor accuracy	ESP32 readings vs. manual audit	IoT sensor dashboard	Weekly audit

5.3 Validation and Testing Plan

Table 11: Validation and Testing Plan

tableheader				
Unit Testing	Each module independently	Jest (NestJS)	All endpoints return expected responses for valid and invalid inputs	Phase 1–3
Integration Testing	SCPMS ↔ Identity Mgmt., Schedule DB, FCM	Postman / Jest supertest	OAuth exchange succeeds; schedule queries return correct time blocks	Phase 2–3
Load / Stress Testing	Backend under simulated peak concurrency	k6 or Apache JMeter	No response degradation; no duplicate reservation conflicts	Phase 2
User Acceptance Testing	End-to-end flow with real students and security staff	On-campus manual sessions	Task completion ≥90%; reservation done in <3 min	Phase 2–3
Regression Testing	Full system after each phase deployment	GitHub Actions CI/CD	All previously passing tests continue to pass	Phase 2–4
Security & Privacy Audit	Encryption, OAuth flows, plate/credential storage	OWASP checklist + manual review	No plain-text PII; tokens expire correctly	Phase 3
Sensor Validation	ESP32 detection accuracy vs. ground truth	On-site audit in Basement 3	≥95% correct detection under realistic conditions	Phase 4

5.4 Continuous Improvement

- **Analytics-driven tuning:** Daily occupancy reports allow Parking Administration to adjust time-slot windows without modifying core logic.
- **UX iteration:** User error rates and incomplete reservation events feed into UX refinement cycles between phases.
- **Modular growth:** New parking areas are added by extending slot configuration — no rewrites of core modules required.
- **IoT-to-ML pathway:** Phase 4 sensor data builds a historical occupancy dataset that enables predictive availability forecasting in future development cycles.

6. Conclusions and Next Steps

This section summarises the key design decisions made throughout Workshop #2, assesses the degree to which the system objectives defined in Section 1 are addressed by the design, and provides the implementation roadmap as the handoff artefact for the next development phase.

6.1 Key Design Decisions

Table 12: Key Design Decisions and Justifications

tableheader		
Deployment model	Mobile-first extension of the existing UD app	No new install friction; leverages existing credentials; accelerates adoption
Hardware strategy	Software-first; IoT deferred to Phase 4	Immediate value delivery without hardware cost; validates the software model before investing in sensors
Data layer	PostgreSQL + Redis	Transactional integrity for concurrent reservations + in-memory speed for <3 s map updates
Authentication	OAuth 2.0 via UD Identity Management System (SSO)	Eliminates duplicate credential management; inherits existing user base
Occupancy control	20-minute TTL auto-release	Prevents passive occupancy; maximises daily vehicle rotation; directly resolves indefinite occupancy problem from Workshop #1
Security alerts	FCM push notifications	Free, reliable, <5 s delivery; no proprietary infrastructure required
Delivery model	4 phased increments	Each phase independently deployable; reduces risk; allows course correction
Privacy compliance	All PII on UD servers; automatic purge; no external processors	Non-negotiable legal requirement (Law 1581/2012); builds institutional trust

6.2 Objective Achievement Assessment

Objective 1 — Operational Optimisation

The WebSocket live map, streamlined reservation flow (<3 min), and mobile check-in collectively eliminate the manual, search-based entry process documented in Workshop #1. The end-to-end transaction KPI provides a quantified success criterion.

Objective 2 — Inventory Efficiency (≥20% utilisation improvement)

Digital cell demarcation removes the spatial disorder root cause (one motorcycle occupying three spaces). The 20-minute TTL auto-release prevents passive occupancy. Both mechanisms maximise Basement 3 capacity without any physical construction.

Objective 3 — Regulatory Compliance

Compliance-by-design — PII on UD servers, no external data processors, automatic purge, OAuth 2.0 — ensures every data flow operates within Law 1581 of 2012. The Security & Privacy Audit in the validation plan provides a formal checkpoint before production deployment.

Objective 4 — Behavioural Control

The Security Alert Subsystem (<5 s FCM alerts), infraction logging linked to plates and credentials, and the proposed sanction mechanism form a three-layer enforcement stack that replaces the current unenforceable honour system with traceable, automated control.

6.3 Implementation Roadmap

Table 13: Implementation Roadmap

tableheader				
Phase 1 — Foundations	Weeks 1–4	UD SSO integration; PostgreSQL schema; first app screens	Student logs in and sees slot list	Schedule IT coordination meeting to define OAuth scope and server provisioning
Phase 2 — Working App	Weeks 5–10	Live map; full reservation flow; TTL release; FCM reminders	Reservation <3 min; map update <3 s	Conduct load test under 06:30–08:30 peak scenario; run UAT pilot with student group
Phase 3 — Control & Alerts	Weeks 11–14	Security panel; violation alerts; duplicate prevention; infraction log	Violation alert in <5 s; concurrent reservations impossible	Security staff 2-hour training session; run full regression test suite
Phase 4 — Physical Sensors	Weeks 15–20	ESP32 sensors per slot; automated detection; admin analytics dashboard	Sensor accuracy ≥95%; search time reduced ≥40%	Coordinate physical installation with Facilities; provision IoT network in Basement 3

6.4 Recommendations for Future Development

- **Multi-basement extensibility:** Define a slot-numbering convention and configuration-driven area management module before Phase 4 to ensure consistency when additional parking areas are onboarded.
- **Predictive occupancy:** Phase 4 sensor data will build a historical dataset that supports a basic time-series model for proactive overflow advisory messaging before demand peaks materialise.
- **Offline synchronisation protocol:** Formalise a conflict resolution strategy (e.g., last-write-wins or CRDT) for cases where an offline check-in conflicts with a simultaneous online reservation.
- **External Overflow Advisory Module:** Develop a formal requirement specification for

the idle-slot detection algorithm and the governance model for overflow access authorisation.

6.5 Final Remarks

The SCPMS design is a precise response to empirically documented operational problems: 79% of users perceive inefficiency, one motorcycle occupies the space of three, and security staff bear the full burden of a process that should be automated. The design resolves all three root causes — information asymmetry, spatial disorder, and absence of prioritisation — through a software-first, phased, compliance-by-design approach that requires zero infrastructure investment in its first three phases.

The system is ready to proceed to Phase 1 upon completion of the initial IT department coordination meeting to establish OAuth integration scope and server provisioning agreements.